



Model Predictive Control - EL2700

State Feedback Control Design

Assignment 1

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
Kungliga Tekniska Högskolan

Gregorio Marchesini, Samuel Erickson Andersson, Alberto Zaupa,
and Mikael Johansson

Background: Python Programming

Throughout the homeworks, we will use Python as our programming language. If you have never used Python before, the best way to start is by looking at the cheatsheet provided in the course syllabus or the free course [Intro to Python programming](#) from the `cvxpy` team. We recommend using VSCode or PyCharm as IDEs for coding, as they automate most of the Python workflow for you. But you are free to use any IDE you might have already installed on your machine.

The following references will get you started, from downloading an IDE to running your first line of code:

- [Installing VSCode and running your first Python program](#)
- [Installing PyCharm and running your first Python program](#)

As you may know (or will soon discover), Python libraries are commonly used to reuse functions that other developers have already written. For example, the `matplotlib` library is widely used to plot all sorts of functions in Python. To use libraries, we need two things: a **package manager**, which takes care of downloading the libraries we need (among other things), and a **virtual environment**, which can be thought of as the physical library where the package manager stores your libraries for you to use.

As a package manager, we encourage you to use `uv`, since it is the package manager we used to build these assignments. You are free to use any other package manager you like, but we strongly believe `uv` is the most reliable and effective option available. Below are instructions on how to install and use `uv`:

- (i) [Installing uv](#) (ii) [Using uv](#) (iii) [Creating a virtual environment with uv](#)

You will find more detailed instructions in the `README.md` file inside the assignment material. If you have never used Python before, this may be a lot of information to take in at once. We suggest taking a few moments to understand the basics of Python through these tutorials, but you will not be required to be an expert programmer. Coding exercises only require you to fill in a few values or create simple functions in a few Python scripts, and you will not be judged on your coding style. Modern AI agents can be a huge help with all of these initial installation steps and will likely resolve most issues you run into. That said, please don't hesitate to reach out if you need help, as we are always happy to assist!

What to submit :

Your submission should be a .ZIP file, named with your Group number (e.g., Group1.zip), containing:

- A PDF document (can be a scanned document, generated from Word, Latex or others) containing the answers and motivation to the questions on the assignment. Make sure to name your document `Assignment.STUDENT1.STUDENT2.pdf`. Answers should be complete but not excessive. You should use figures of the plots you receive from the scripts to help yourself explain and corroborate your arguments when needed. Any mathematical passage or explanation you might need to answer a question should be reported in your PDF.
- The *.py code files with a working `task1.py` (i.e., needs to run in one click) that shows your solution on the plot output.

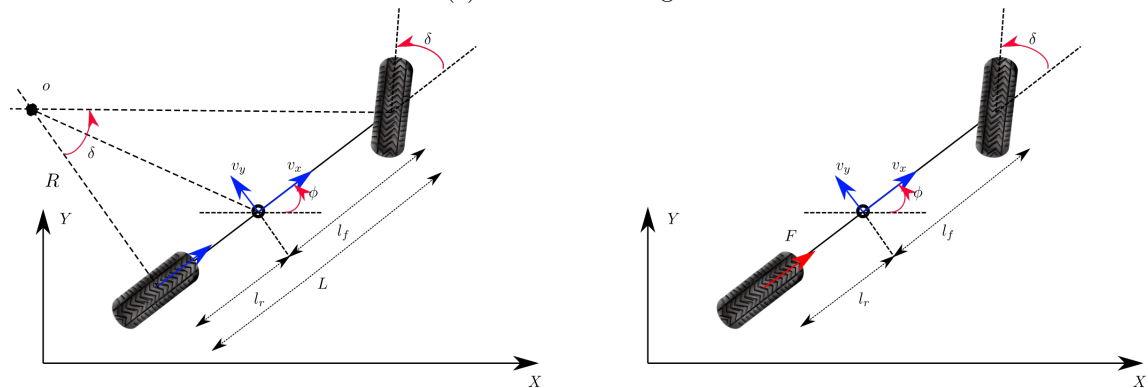
Good Luck! And, if in doubt, ask!

Introduction

During the course, we will explore vehicle dynamics as our reference system for experimenting with Pole Placement, the Linear Quadratic Regulator (LQR), and Model Predictive Control (MPC). This assignment is the first in a series that will take you from being a rookie pilot to a professional racing driver. In this assignment, we will design a discrete-time linear state-feedback controller for a simple vehicle model. To do this, you will **compute a discrete-time model** describing the state evolution between sampling instants, and use this model to **design a linear state-feedback controller** that achieves the desired control performance.



(a) Valentino at Mugello



(b) Graphical representation of the kinematic bicycle model

Figure 1: Overall caption for both figures

To complete the task, carefully look into the Python files that are part of the assignment and the classes that they implement. The code entry point is **task1.py**, but you will have to **complete parts of the LinearCarModel and Controller classes**.

Model Dynamics

A common first-pass vehicle model for control design is the **bicycle model** (Figure 1b), consisting of two wheels separated by wheelbase $L = l_r + l_f$, where l_f, l_r are the distances from the center of mass to the front and rear wheels, respectively. The model consist of the following variables.

	Description		Description
X, Y	Inertial position of center of mass	δ	Front wheel steering angle
ϕ	Heading relative to inertial X -axis	R	Turning radius
v_x, v_y	Forward and lateral velocity (body frame)	o	Center of rotation
		F	Rear-wheel traction force

Table 1: Model variables (see Figure 1b).

Intuitively, for constant v_x , a fixed steering angle δ makes the vehicle pivot about a center of rotation o , found geometrically as the intersection of the lines normal to each wheel. Basic trigonometry on the wheelbase gives

$$L = R \tan \delta \Rightarrow R = \frac{L}{\tan \delta}. \quad (1)$$

Using the rigid-body kinematics, we can find that if the vehicle is driving at constant speed v_x and with a given steering angle δ , then the *heading rate* of $\dot{\phi}$, (i.e., the rotational speed around the center of rotation o) is given by

$$\dot{\phi} = \frac{v_x}{R} = v_x \tan(\delta) \cdot \frac{1}{L} \quad [rad/s] \quad (2)$$

while the lateral velocity v_y is given by:

$$v_y = v_x \tan(\delta) \cdot \frac{l_r}{L} \quad [m/s] \quad (3)$$

In other words, if you are fast, then small steering angles δ are sufficient to engage you in fast rotations. If you instead are slow, you might need a larger steering angle to turn fast. Rear-wheel traction force F produces longitudinal acceleration such that:

$$\dot{v}_x = \frac{F}{m}. \quad (4)$$

Assuming $\tan \delta \approx \delta$ (valid roughly for $\delta < 30^\circ$), the five-state bicycle model is

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{v}_x \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} v_x \cos \phi - v_y \sin \phi \\ v_x \sin \phi + v_y \cos \phi \\ \frac{1}{m} F \\ v_x \delta \frac{1}{l_r + l_f} \end{bmatrix} \quad (5)$$

with the kinematics relation

$$v_y = v_x \delta \frac{l_r}{l_r + l_f}$$

Why this isn't enough as an abstraction: Since $v_y \propto \delta$ directly, a step change in δ causes an instantaneous jump in lateral velocity, which is mostly unrealistic for reasonable speed (this is why kinematics is now enough to have a realistic model). To fix this physical inconsistency, we treat the *steering rate* $u = \dot{\delta}$ as the input instead of δ itself, promoting δ to a state. Differentiating v_y and $\dot{\phi}$ expressions (product rule) gives

$$\dot{v}_y = (\dot{v}_x \delta + \dot{\delta} v_x) \frac{l_r}{l_r + l_f}, \quad \dot{r} \equiv \ddot{\phi} = (\dot{v}_x \delta + \dot{\delta} v_x) \frac{1}{l_r + l_f}, \quad (6)$$

and yields the full seven-state model:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\phi} \\ \dot{v}_x \\ \dot{v}_y \\ \dot{r} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} v_x \cos \phi - v_y \sin \phi \\ v_x \sin \phi + v_y \cos \phi \\ r \\ \frac{1}{m} F \\ (\dot{\delta} v_x + \dot{v}_x \delta) \frac{l_r}{l_r + l_f} \\ (\dot{\delta} v_x + \dot{v}_x \delta) \frac{1}{l_r + l_f} \\ u \end{bmatrix}. \quad (7)$$

This system is a nonlinear system with seven states and two inputs: rear traction force F and steering rate u (nonlinearities come from $\sin, \cos$ and input-state products). For this assignment, we use a **linearized lateral-dynamics** model built from the above to start getting a feeling about the model and use the control techniques we have learned in class.

Design Task

Before competing at any level, riders must pass a lane-change test to demonstrate they can safely avoid road obstacles. Vehicles losing parts mid-race are not uncommon, and following competitors must be able to react. In this assignment, we use a simplified linear lateral model to perform a lane-change maneuver around an obstacle (Figure 2). Assume the vehicle moves at constant longitudinal velocity ($\dot{v}_x = 0$) with heading held near zero ($\phi \approx 0$). Under these assumptions, the nonlinear model reduces to the linear lateral dynamics:

$$\dot{Y} = v_y, \quad \dot{v}_y = v_x \delta \frac{l_r}{l_r + l_f}, \quad (8)$$

or, in state-space form with $\mathbf{x} = [Y \ v_y]^T$ and input $u = \delta$,

$$\dot{\mathbf{x}}(t) = A_c \mathbf{x}(t) + B_c u(t), \quad A_c = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ v_x \frac{l_r}{l_r + l_f} \end{bmatrix}. \quad (9)$$

The longitudinal dynamics remains an *uncontrolled* single integrator,

$$\dot{X} = v_x, \quad (10)$$

so with initial condition $X(0) = 0$, position grows linearly: $X(t) = v_x t$.

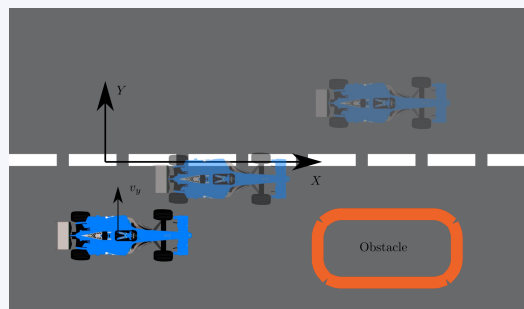


Figure 2: Overtake manoeuvre.

You should now be ready to answer the following questions:

Question 1

Inside the class `LinearCarModel` implement the continuous model dynamics above in the function `_compute_system_matrices`. You can access the parameters of the vehicle using the `self.` notation. For example, the mass of the vehicle is available as `self.mass`

Question 2

As you may have noticed, the continuous-time model is a double integrator. Compute the discrete model matrices A, B from the continuous time matrices A_c, B_c and compare the result with the numerical values returned by `c2d` - are they the same? Now, observe that we can write the discrete-time model as

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + Bu_k \quad y_k = C\mathbf{x}_k + Du_k \quad (11)$$

Question 2

Compute the transfer function of the continuous-time model from the control input u to the output y , considering $C = [1 \ 0]$ and $D = 0$. How many poles and zeros does the system have? Where are they located? How many poles and zeros do you expect for the discrete-time model, and where should they be located? Was your intuition right?

Note: For this question, you can use Matlab or any other tool you are comfortable with. In Python, the `control` library provides the same functionality as Matlab (classes `StateSpace`, `TransferFunction` and method `control.pzmap`), but requires `slycot` to be installed—already available on the Virtual Machine. The method `poles_zeros` in the `LinearCarModel` class implements this functionality.

Question 4

We are finally ready to design our controller for the change of lane. We will assume the road is 4 m with the centerline located at the lateral coordinate $Y = 0$. The initial state of the car is $\mathbf{x}_0 = [-1.0 \ 0.0]^T$ corresponding to the center of the lower lane in Figure 2. In order to change lane and avoid the obstacle (orange oval in Figure 2), we want to reach the reference state $\mathbf{x}_{\text{ref}} = [1.0 \ 0.0]^T$ corresponding to the middle of the upper lane. Based on the discretized linear model in (11), derive a state feedback controller of the form

$$u_k = -L(\mathbf{x}_k - \mathbf{x}_{\text{ref}}) \quad (12)$$

where L is the state feedback gain, designed by placing the poles of the closed-loop poles at the desired location using the function `place` in the function `get_closed_loop_gain`. Implement this controller in the function `control_law` and make sure the following constraints are satisfied.

- Since the obstacle is 200 m ahead and you are driving at $v_x = 20$ m/s we need to reach at least 75% of the reference within 10 s to be sure the obstacle is avoided.
- To not give the impression that we don't know what we are doing, you should avoid sharp driving for this simple scenario. Thus, the maximum steering rate u should satisfy $|u| < 0.25$ and the maximum lateral velocity should be $|v_y| < 1$ m/s.
- You should **not overshoot** more than 1 m to avoid going offroad!!

Question 5

After several ground tests, we realized that our model presents some dynamical error that we approximate as a constant disturbance in the lateral velocity of 2 cm/s. This effect is simulated in the function `set_disturbance`. To fix this, we design an integral action

according to

$$u_k = -L(\mathbf{x}_k - \mathbf{x}_{ref}) - K_i i_k \quad i_{k+1} = i_k + h \cdot (p_k - p_{ref}) \quad (13)$$

where p_k is p_{X_k} and p_{ref} the desired reference position. Implement the integral action (13) in the functions `update_integral` and `control_law`, under the `Controller` class. Design the feedback such that you don't go offroad and you reach the reference within 30 s, respecting the initial given constraints on the input and velocity given in Q4. The function `activate_integral_action` can then be used to set an integral gain and system sampling time. **Note:** constraints in Q4 also apply here.